ESP32-P4 Insect Counting Trap — Firmware Guide

Status: Implementation complete — Phase 1 (WiFi + ESPDet-Pico inference) **Date:** April 2026 **Target:** ESP32-P4NRW32, dual HP RISC-V @ 400 MHz, 32 MB on-package PSRAM **Camera:** OV5647 (Raspberry Pi Camera v1 sensor), MIPI CSI-2 1-lane **Source:** `firmware-esp32p4/` (ESP-IDF ≥ 5.3, C++17)

Overview

This firmware implements a low-cost, low-power insect counting trap on the **ESP32-P4NRW32** SoC. Unlike the CM5-based trap, it does **not** store images in routine operation — it counts insects and transmits compact detection events over WiFi (or LoRaWAN in Phase 3).

The primary output is a time-series count: one ~60-byte JSON event per detection, POSTed to a configurable HTTP server. A full day of busy-trap activity fits in a few KB of data.

Why count-only? For large-scale ecological monitoring — 50 to 200+ traps per study area — the scientific output is a detection time-series, not a video archive. Eliminating image storage reduces per-unit cost, power consumption, and data infrastructure requirements by an order of magnitude.

Hardware

Bill of materials

Component	Part	Notes
SoC module	M5Stack Stamp-P4	ESP32-P4NRW32, 32 MB PSRAM, MIPI CSI connector
Camera	OV5647 MIPI module	Raspberry Pi Camera v1 sensor, 5 MP
PIR sensor	HC-SR501 or equiv.	Active-high output on GPIO5
Radio (WiFi)	M5Stack Stamp-AddOn C6	ESP-Hosted SPI to Stamp-P4
Radio (LoRa)	SX1262 breakout on SPI	Phase 3 — not yet implemented
Battery	LiPo 3000–5000 mAh	Or 2–5 W solar + 1000 mAh

GPIO pinout (defaults, all configurable via NVS)

Signal	GPIO	Direction	Notes
OV5647 PWDN	6	Output	Active-high power-down

Signal	GPIO	Direction	Notes
OV5647 SCCB SCL	8	Output	I2C master @ 100 kHz
OV5647 SCCB SDA	9	Bidirectional	I2C master
OV5647 MIPI D0+/-	CSI lane 0	Input	Differential, PCB routing
OV5647 MIPI CLK+/-	CSI clock	Input	Differential, PCB routing
PIR OUT	5	Input	POSEDGE → LP core wakeup

Camera pipeline

The OV5647 is a 5 MP CMOS sensor (also used in the Raspberry Pi Camera Module v1). Here it is driven in **MIPI CSI-2 1-lane mode** at 640×480 RAW10, ~30 fps.

The P4's hardware ISP receives the RAW10 MIPI stream and produces **224×224 YUV420** output in DMA SRAM — the exact input resolution of the ESPDet-Pico model — with no CPU involvement in the resize or demosaic steps.

```
OV5647 sensor          ESP32-P4 hardware blocks
──────────────         ────────────────────────────────────────────
640×480 RAW10   MIPI   ┌──────────┐  RAW10   ┌──────────┐  224×224
1-lane 200 Mbps ──────► │ MIPI     │ ────────► │ ISP      │  YUV420
                        │ D-PHY    │           │ AWB/AEC  │ ────────► DMA frame buffer
                        └──────────┘           │ demosaic│          (internal SRAM)
                                               │ crop+    │
                                               │ scale    │
                                               └──────────┘
```

The ISP performs automatic white balance, auto exposure, noise reduction, and demosaicing in hardware. Cropping the 640×480 area to 224×224 (centre crop) is also done in the ISP at zero CPU cost.

Frame buffer size: 224 × 224 × 1.5 = **75,264 bytes per buffer** — fits comfortably in the 768 KB internal SRAM with two ping-pong buffers allocated.

Inference pipeline

Model: ESPDet-Pico

Property	Value
Architecture	Ultralytics YOLOv11-derived, single-class
Parameters	0.36 M

Property	Value
Input resolution	224×224 RGB888
Quantization	INT8 (ESP-PPQ) → `.espdl` format
Inference time (P4)	~55 ms (~18 FPS)
Framework	ESP-DL ≥ 2.0.0
Storage	`model` flash partition (4 MB, offset 0x290000)

Weights are loaded from flash into PSRAM at startup. Hot activations are placed in internal SRAM by ESP-DL's memory planner to avoid PSRAM bandwidth contention during inference.

Pixel Processing Accelerator (PPA)

Before inference, the frame buffer (YUV420 in DMA SRAM) is passed through the P4's **Pixel Processing Accelerator** — a hardware block that converts colour spaces and performs scale/rotate/mirror operations with zero CPU overhead.

```
Frame buffer           PPA (hardware)           inference_buf
224×224 YUV420  ──────► YUV420 → RGB888  ──────► 224×224 RGB888
(DMA SRAM)             (zero CPU cost)           (static SRAM, 150 KB)
```

The capture buffer is returned to the pool **immediately after PPA completes**, before ESP-DL inference starts. CameraTask can begin the next capture while InferenceTask runs on the static `inference_buf` copy — this is the key zero-copy pipeline optimisation.

Inference runtime

ESP-DL runs ESPDet-Pico in `RUNTIME_MODE_MULTI_CORE` mode: the two HP RISC-V cores share the Conv2D workload, each core running interleaved filter computations. The PIE/Xai SIMD extension provides 16 parallel INT8 MACs per cycle per core.

FreeRTOS task architecture

Core allocation

```
Core 0 (PRO_CPU)           Core 1 (APP_CPU)
─────────────────          ─────────────────
CameraTask   pri=6         InferenceTask  pri=5
RadioTask    pri=3
PowerTask    pri=1

LP RISC-V core (always on during deep sleep)
  ulp_pir_monitor — PIR GPIO interrupt handler
```

Inference is pinned to Core 1 so that `RUNTIME_MODE_MULTI_CORE` spawns the ESP-DL helper task onto Core 0 — the two cores share Conv2D without competing on the same core for scheduler time.

Task inventory

Task	Core	Priority	Stack	Role
CameraTask	0	6	6 KB	OV5647 SCCB init, CSI controller, frame buffer pool
InferenceTask	1	5	12 KB	PPA colour convert, ESPDet-Pico, emit DetectionEvents
RadioTask	0	3	8 KB	WiFi STA, HTTP POST, NVS ring buffer retry
PowerTask	0	1	4 KB	Idle timer, orderly shutdown, deep sleep entry

All tasks, queues, and semaphores use **static FreeRTOS allocation** (`StaticTask_t` , `StaticQueue_t`). There is no heap fragmentation and stack overflows are detectable at link time.

CameraTask

Owns the camera controller lifecycle and the frame buffer pool.

At startup it initialises the OV5647 over SCCB (I2C), writes the 640×480 MIPI 1-lane register table, and starts the P4 CSI controller. Two DMA frame buffers are allocated in internal SRAM and pre-loaded into `g_buffer_pool_queue` .

The capture loop pulls a free buffer from the pool, submits it to the CSI controller, and waits for the next free buffer. The ISR callback fires on frame completion and pushes the buffer pointer to `g_frame_queue` . If InferenceTask has not consumed the previous frame, the ISR drops the new frame and returns the buffer to the pool immediately — never blocking.

InferenceTask

The compute bottleneck. Runs continuously while frames are available; blocks on `g_frame_queue` between events, allowing the system to light-sleep.

1. Pop `FrameBuffer*` from `g_frame_queue`
2. PPA: YUV420 → RGB888 into static `inference_buf` (blocking, ~2 ms)
3. **Return buffer to pool immediately** — CameraTask can capture next frame
4. ESP-DL inference on `inference_buf` (~55 ms, MULTI_CORE)
5. Postprocess: filter by confidence threshold, emit `DetectionEvent` s

RadioTask

Consumes `DetectionEvent` s from `g_detection_queue` . Maintains a WiFi STA connection with automatic reconnect. On success, serialises the event as JSON and POSTs to `server_url` . On failure, writes the event to an NVS ring buffer (256 events) for retry when connectivity is restored.

Runs at priority 3 — below InferenceTask — so radio latency never stalls the inference pipeline.

PowerTask

Monitors the time since the last `DetectionEvent`. When no detection has been processed for `idle_timeout_s` seconds **and** both queues are empty:

1. Sets `g_shutdown_requested = true` and signals CameraTask to stop
2. Waits for `g_frame_queue` to drain (InferenceTask finishes current frame)
3. Waits for `g_detection_queue` to drain (RadioTask sends or buffers all events)
4. Loads the LP core ULP program (`ulp_pir_monitor`)
5. Calls `esp_deep_sleep_start()` — the only call site in the firmware

PowerTask is the **only task that ever calls `esp_deep_sleep_start()`**. This prevents race conditions where two tasks simultaneously initiate sleep.

Queue design

All queues are statically allocated. Producers drop rather than block on full queues — the pipeline degrades gracefully under load.

Queue	Item type	Depth	Producer	Consumer	Full policy
`g_frame_queue`	`FrameBuffer*`	2	Camera ISR	InferenceTask	Drop frame
`g_buffer_pool_queue`	`FrameBuffer*`	2	InferenceTask	CameraTask	Never full (depth = pool size)
`g_detection_queue`	`DetectionEvent`	16	InferenceTask	RadioTask	Drop event, log count

`g_frame_queue` depth 2 means InferenceTask can be at most 1 frame behind. If inference takes longer than the capture interval, the oldest unprocessed frame is silently dropped. At ~55 ms per frame and ~55 ms inference time this is a soft constraint — occasional frame drops are expected and acceptable.

Memory layout

```
Internal SRAM (768 KB total — DMA-capable, zero wait-state)
┌──────────────────────────────────────────────────────┬──────────┐
│ FreeRTOS kernel + OS overhead                        │  ~80 KB  │
│ Task stacks (4 tasks × ~8 KB average)                │  ~32 KB  │
│ Static TCBs + queue storage                          │   ~8 KB  │
│ Frame buffer A  (224×224 YUV420, DMA)                │  ~75 KB  │
│ Frame buffer B  (224×224 YUV420, DMA)                │  ~75 KB  │
│ inference_buf   (224×224 RGB888, PPA destination)  | ~150 KB  |
│ ESP-DL hot activations (MALLOC_CAP_INTERNAL)       | ~100 KB  |
│ Driver buffers, WiFi, headroom                     | ~248 KB  |
```

```
└──────────────────────────────────────────────────────┴──────────┘

PSRAM (32 MB on-package octal SPI)
┌──────────────────────────────────────────────────────┬──────────┐
│ ESPDet-Pico model weights (.espdl)                   │   ~2 MB  │
│ ESP-DL cold activations + work buffers               │   ~2 MB  │
│ Available for future use                             │  ~28 MB  │
└──────────────────────────────────────────────────────┴──────────┘
```

DMA constraint: `g_frame_queue` buffers and `inference_buf` are allocated with `MALLOC_CAP_DMA | MALLOC_CAP_INTERNAL`. PPA source and destination must be DMA-accessible — PSRAM cannot be used for these buffers.

Power states

```
          PIR POSEDGE → LP core ISR
                  │
┌─────────────────▼──────────────────┐
│          DEEP SLEEP                │ LP core active, HP cores off
│   ulp_pir_monitor running          │ ~1–5 mA total
└────────────────────────────────────┘
                  │ ulp_lp_core_wakeup_main_processor()
┌─────────────────▼──────────────────┐
│            BOOTING                 │ HP cores, NVS init, task start
│          ~10–50 ms                 │ ~150–200 mA
└────────────────────────────────────┘
                  │ pipeline ready
┌─────────────────▼──────────────────┐
│            RUNNING                 │ Camera + inference loop
│  ESP_PM_CPU_FREQ_MAX held          │ ~200–400 mA during inference
│  light-sleep between frames        │ ~10–20 mA between frames
└────────────────────────────────────┘
          │ detection    │ no detection for idle_timeout_s
          │              │
┌─────────▼──────────┐   ┌──────▼──────────────────────┐
│  TRANSMIT          │   │         DRAINING            │
│ ~300mA WiFi        │   │ queues drain, radio done    │
│  ~40mA LoRa        │   └─────────────────────────────┘
└────────────────────┘                │
          │                           │ drained
          │                           ▼
          └──────────► load LP ULP → DEEP SLEEP
```

Estimated power budget

State	Current	Typical duty
Deep sleep (LP core + PIR)	~1–5 mA	~99% at low insect activity
Boot + init	~150–200 mA	~50 ms per wakeup
Camera + inference running	~200–400 mA	~55 ms per detection cycle

State	Current	Typical duty
WiFi transmit	~250–300 mA	~500 ms per event
LoRa transmit (future)	~40 mA	~50–500 ms per event

On a 3000 mAh LiPo at 1% duty cycle: **weeks to months** of runtime without solar. A 2–5 W solar panel sustains indefinite continuous operation.

This is fundamentally different from the CM5 trap which draws 4–8 W continuously regardless of activity.

LP core ULP program

The LP RISC-V core runs `ulp_pir_monitor.c` while the HP cores are off. It consumes approximately 1 µA of active current (LP core only, with GPIO interrupt).

```
// ulp_pir_monitor.c — simplified outline
void LP_CORE_ISR_ATTR ulp_lp_core_lp_io_intr_handler(void) {
    // 500 ms debounce — ignore rapid re-triggers
    if (elapsed_ms < DEBOUNCE_MS) { lp_core_intr_umask_all(); return; }

    // Confirm GPIO still high (reject noise spikes)
    if (lp_io_get_level(pir_gpio_num) == 0) { lp_core_intr_umask_all(); return; }

    ulp_wakeup_count++;
    ulp_lp_core_wakeup_main_processor();  // boot HP cores
    lp_core_intr_umask_all();             // re-arm for next event
}
```

Shared RTC memory variables (survive deep sleep):

Variable	Written by	Read by	Purpose
`ulp_pir_gpio_num`	HP / PowerTask	LP core	GPIO pin to monitor
`ulp_wakeup_count`	LP core	HP / telemetry	Monotonic wakeup counter
`ulp_last_motion_ts`	LP core	LP core	Debounce timestamp

Detection event format

Each confirmed detection is serialised as compact JSON and POSTed to `server_url`:

```
{"trap":1,"ts":1743000042,"cls":0,"conf":87,"cnt":1}
```

Field	Type	Description
`trap`	uint16	Trap ID (configured in NVS)
`ts`	int64	Unix timestamp in seconds (NTP-synced if available)
`cls`	uint8	Species class index (0 = insect — single-class model)
`conf`	uint8	Model confidence 0–100
`cnt`	uint8	Detection count (always 1; future: burst coalescing)

At ~55 bytes per event, a day of busy-trap activity at 1 detection per minute produces ~80 KB of data — negligible compared to the CM5's image archive.

NVS ring buffer

When the WiFi uplink is unavailable, RadioTask writes events to an NVS ring buffer (capacity: 256 events, ~14 KB). On reconnection, buffered events are flushed in order before new events are transmitted. The ring buffer uses overwrite-oldest semantics when full.

Configuration (NVS)

All runtime parameters are stored in the `trap_cfg` NVS namespace and loaded at startup. Compiled-in defaults are used for any key not present in NVS.

Key	Type	Default	Description
`trap_id`	u16	1	Unique trap identifier
`location`	string	`"unset"`	Location name (metadata only)
`conf_thresh`	u32 (float bits)	0.45	Minimum detection confidence
`idle_timeout`	u32	30	Seconds without detection before deep sleep
`pir_gpio`	u8	5	GPIO connected to PIR sensor output
`radio_mode`	u8	0 (WiFi)	0=WiFi, 1=LoRa
`wifi_ssid`	string	`""`	WiFi network name
`wifi_pass`	string	`""`	WiFi password
`server_url`	string	`http://...`	HTTP POST endpoint for detection events
`lora_freq`	u32	868100000	LoRa carrier frequency in Hz (EU868)
`lora_dev_eui`	string	zeros	LoRaWAN device EUI (hex)
`lora_app_key`	string	zeros	LoRaWAN application key (hex)

Key	Type	Default	Description
`sample_rate`	u32	0	1-in-N JPEG sampling for QC (0 = disabled)

Provisioning via NVS partition image

```
# Create config CSV
cat > nvs_config.csv << 'EOF'
key,type,encoding,value
trap_cfg,namespace,,
trap_id,data,u16,42
wifi_ssid,data,string,MyNetwork
wifi_pass,data,string,MyPassword
server_url,data,string,http://192.168.1.100:8080/api/detections
idle_timeout,data,u32,30
EOF

# Generate partition image
python $IDF_PATH/components/nvs_flash/nvs_partition_generator/nvs_partition_gen.py \
    generate nvs_config.csv nvs.bin 0x6000

# Flash NVS partition
esptool.py --port /dev/ttyUSB0 write_flash 0x9000 nvs.bin
```

Flash layout

```
Offset      Size    Name        Contents
──────────  ──────  ──────────  ────────────────────────────────────────
0x0000      32 KB   Bootloader  ESP-IDF second-stage bootloader
0x9000      24 KB   nvs         NVS partition (trap config + radio buffer)
0xF000       4 KB   phy_init    RF calibration data
0x10000    2.5 MB   factory     Application binary (app + ULP binary)
0x290000    4 MB    model       ESPDet-Pico .espdl model weights
0x690000   1.5 MB   storage     SPIFFS (logs, sampled JPEGs if enabled)
```

Build and flash

Prerequisites

- ESP-IDF **≥ 5.3.0** (P4 CSI controller API and LP core support)
- Python 3.10+

```
# Install ESP-IDF
git clone --recursive https://github.com/espressif/esp-idf ~/esp/esp-idf
```

```
cd ~/esp/esp-idf && git checkout v5.3 && ./install.sh esp32p4
source ~/esp/esp-idf/export.sh
```

Build

```
cd firmware-esp32p4
idf.py set-target esp32p4          # once per checkout
idf.py update-dependencies         # fetch managed components (needs internet)
idf.py build
```

Flash

```
# Flash application
idf.py -p /dev/ttyUSB0 -b 921600 flash

# Flash ESPDet-Pico model (download from espressif/esp-dl releases)
esptool.py --port /dev/ttyUSB0 --baud 921600 \
    write_flash 0x290000 espdet_pico_224_int8.espdl

# Monitor
idf.py -p /dev/ttyUSB0 monitor
```

Expected boot log

```
I (342)  main: Boot reason: 0 (power-on)
I (344)  main: PM configured: max=400 MHz, min=40 MHz, light-sleep=on
I (346)  trap_cfg: No NVS namespace found — using compiled defaults
I (348)  main: Config: trap_id=1, conf_threshold=0.45, idle=30 s
I (350)  main: Free SRAM: 512 KB | Free PSRAM: 30720 KB
I (352)  main: Starting tasks…
I (354)  ov5647: I2C bus init OK
I (362)  ov5647: OV5647 detected (chip ID 0x5647)
I (420)  ov5647: Writing 640×480 MIPI 1-lane register table…
I (428)  ov5647: OV5647 configured: 640×480 RAW10 MIPI 1-lane
I (430)  camera: CSI controller started — 224×224 YUV420
I (432)  inference: Loading ESPDet-Pico from flash partition 'model'…
I (890)  inference: ESPDet-Pico loaded OK
I (892)  inference: Entering inference loop
I (894)  radio: Starting radio task (mode=WiFi)
I (896)  main: All tasks started — FreeRTOS scheduler running
I (898)  main: ai-trap v0.1.0 | trap_id=1 | ESP32-P4 + OV5647
```

Source file index

File	Purpose
`main/pipeline.h`	Shared types: `FrameBuffer`, `DetectionEvent`, `StaticQueue<>`, `Task<>`, `PmLock`, global queue declarations
`main/trap_config.h`	`TrapConfig` struct, NVS load/save
`main/ov5647.h` / `.cpp`	OV5647 SCCB init, 640×480 MIPI register table, stream control
`main/camera_task.h` / `.cpp`	`CameraTask`: CSI controller, frame buffer pool, ISR callback
`main/inference_task.h` / `.cpp`	`InferenceTask`: PPA convert, ESP-DL model run, postprocess
`main/radio_task.h` / `.cpp`	`RadioTask`: WiFi STA, HTTP POST, NVS ring buffer
`main/power_task.h` / `.cpp`	`PowerTask`: idle timer, queue drain, deep sleep entry
`main/app_main.cpp`	Startup sequence, queue definitions, task launch
`ulp/ulp_pir_monitor.c`	LP core program: PIR GPIO interrupt → wakeup HP cores
`CMakeLists.txt`	ESP-IDF top-level project
`sdkconfig.defaults`	P4 target, PSRAM, PM, LP core, WiFi tuning
`partitions.csv`	Flash layout: app + model + NVS + SPIFFS
`idf_component.yml`	Managed components: `esp-dl`, `esp-detection`, `esp_cam_sensor`

Comparison with CM5 trap

	ESP32-P4 (this firmware)	Luckfox Pico Zero	Raspberry Pi CM5
Primary output	Count events	JPEG crops + counts	JPEG crops + counts
Inference model	ESPDet-Pico 224×224	YOLO11n 320×320	YOLO11n 320×320
Inference speed	~18 FPS	~2–3 FPS	~20 FPS
Active power	~0.5–1 W	~0.5 W	4–8 W
Sleep power	~5–15 mW	~50–100 mW	~200 mW+
Battery life (3000 mAh)	Weeks–months	Days–weeks	Hours–days
Radio	C6 addon (WiFi 6/BLE)	AIC8800DC built-in	Built-in
OS	FreeRTOS (ESP-IDF)	Linux (busybox)	Linux (Debian)
Per-unit cost	~£30–50	~£20–30	~£100–150
Image storage	Optional (sampling only)	Yes (every detection)	Yes (every detection)

Phase roadmap

Phase	Scope	Status
1	ESPDet-Pico inference + WiFi HTTP + NVS buffer + PIR sleep	**Complete**
2	NTP wall-clock timestamps, 1-in-N JPEG sampling, SPIFFS logging	Planned
3	SX1262 LoRaWAN radio path, TTN integration, OTA via WiFi	Planned
4	Multi-species model, field enclosure validation, CM5 accuracy comparison	Planned

---

Known limitations

Issue	Severity	Mitigation
ESPDet-Pico accuracy on target species unvalidated	High	Phase 1 bench test against labelled insect images
Timestamps use `esp_timer_get_time()` (since boot, not wall clock)	Medium	Add NTP sync on WiFi connect in Phase 2
LoRa (SX1262) RadioTask path not implemented	Medium	WiFi sufficient for initial deployments near infrastructure
OTA firmware update requires USB connection	Low	Use USB reflash at field station; LoRa OTA in Phase 3
ESP32-P4 silicon revision v1.0 vs v1.3 (pin change, same part number)	Medium	Verify revision before bulk order — see esp32-p4-ai-evaluation.md
`.espdl` model format not forward-compatible across major ESP-DL versions	Low	Pin ESP-IDF + ESP-DL versions in `idf_component.yml`

Related documents

- esp32-p4-insect-trap-architecture.md — System-level design, network options, experimental roadmap, cost model
- esp32-p4-runtime-architecture.md — FreeRTOS task design, queue layout, memory map, C++ patterns
- esp32-p4-ai-evaluation.md — ESP32-P4 hardware capabilities and AI benchmark data